

TRƯỜNG ĐẠI HỌC FPT



FPT UNIVERSITY

LAB211 – OOP WITH JAVA LAB

**EMPLOYEE PAYROLL MANAGEMENT
SYSTEM REPORT**

INSTRUCTOR'S NAME: LÊ VÕ MINH THƯ

Submission Day: 23 June 2026

SE203056	Nguyễn Phú Khương
----------	-------------------

1. Project Overview

The "Employee Payroll Management System" project (J1.L.P0037) is a console-based application developed in Java following the Object-Oriented Programming (OOP) approach. It manages employee information and calculates the monthly payroll. The system also includes a role-based authorization layer: users must log in, and each role (Admin or Staff) is granted access to a specific set of functions. Employee records are stored in a text file (employees.txt), and the system supports loading, adding, updating, removing, searching, payroll calculation, displaying and saving data safely.

Main Features:

#	Feature	Target	Access
-	Login (role-based authentication)	Security	All users
1	Load employee data from file	System / File	Admin, Staff
2	Add a new employee	Employee	Admin
3	Update employee information	Employee	Admin
4	Remove an employee by ID	Employee	Admin
5	Search employees by attribute	Employee	Admin, Staff
6	Calculate monthly payroll (active only)	Payroll	Admin, Staff
7	Display employee list	Employee	Admin, Staff
8	Save data to file	System / File	Admin
9	Quit program (confirm saving)	System	Admin, Staff

Access Control (Authorization):

Each role is mapped to a set of allowed features. The check is polymorphic: the menu asks the account object whether it can access a feature, without any if-else on the role.

Role	Allowed Features	Description
Admin	1, 2, 3, 4, 5, 6, 7, 8, 9	Full access (all functions)
Staff	1, 5, 6, 7, 9	Read-only (load, search, payroll, display, quit)

Data Constraints:

Attribute	Constraint	Valid example
id	Format ^E\d{3}\$ (E001..E999), non-empty, unique	E001
name	Non-empty	David Miller
role	One of: Developer / Tester / Manager / HR	Developer
baseSalary	Positive double (> 0)	1300
workingDays	Integer in range 0 - 26	21
bonus	Double >= 0	250

status	active or inactive	active
--------	--------------------	--------

Salary formula (not given by the assignment - self-defined and justified):

$\text{salary} = \text{baseSalary} / 26 * \text{workingDays}$ (salary by working days)
 + bonus
 + roleAllowance() (role-based allowance - polymorphic)

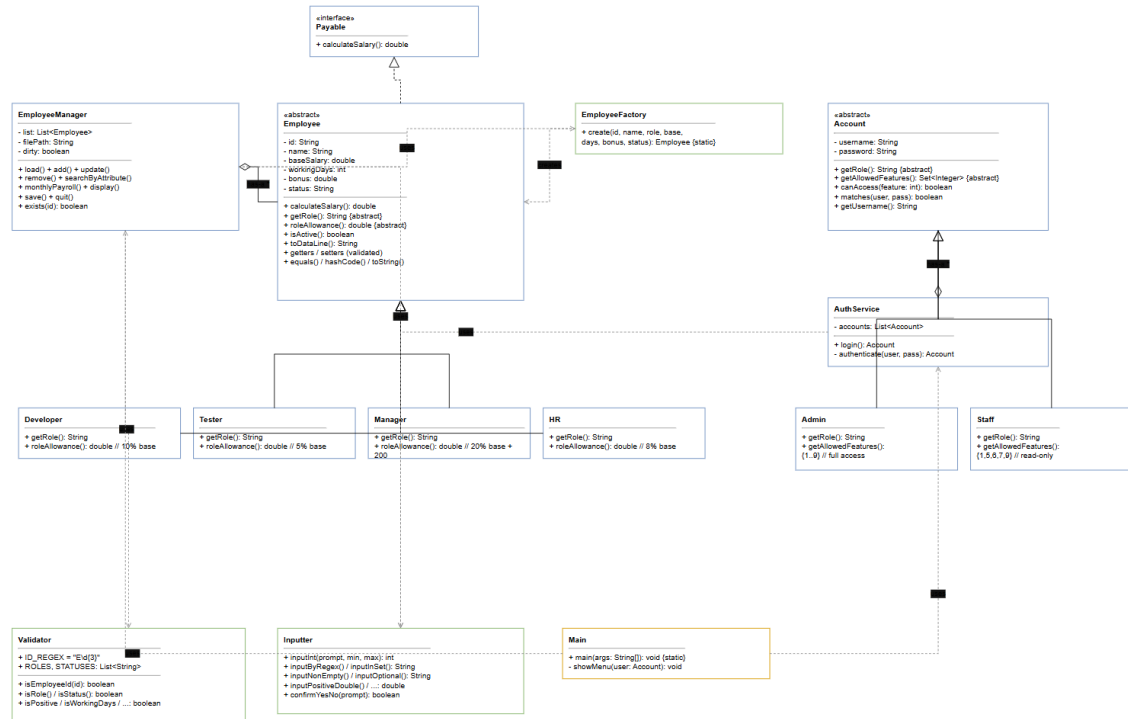
roleAllowance: Developer 10%*base | Tester 5%*base
 Manager 20%*base + 200 | HR 8%*base

Note: 26 is the standard maximum working days defined in the assignment. The monthly payroll (Feature 6) only sums salaries of employees whose status is active.

2. Class Diagram & UML

2.1. Class Diagram

The Class Diagram below illustrates the relationships between all classes in the system (employee model + authorization model).



Class Relationships:

Relationship	From	To	Type
Implementation	Employee	Payable	implements
Inheritance	Developer / Tester / Manager / HR	Employee	extends

Inheritance	Admin / Staff	Account	extends
Association	EmployeeManager	List<Employee>	has-a
Association	AuthService	List<Account>	has-a
Dependency	EmployeeManager	EmployeeFactory	uses
Dependency	Main	EmployeeManager / AuthService	uses

2.2. Use Case Diagram

The Use Case Diagram shows all system functions from the user's perspective, including login and role-based access.



3. Project Structure

The project follows a layered architecture (Decomposition by class):

```
EmployeePayrollManagementSystem/
|-- src/
|   |-- model/                <-- Data Layer
|   |   |-- Payable.java      (interface - calculateSalary)
|   |   |-- Employee.java     (abstract base class)
|   |   |-- Developer.java    (extends Employee)
|   |   |-- Tester.java       (extends Employee)
|   |   |-- Manager.java      (extends Employee)
|   |   |-- HR.java           (extends Employee)
|   |   |-- Account.java      (abstract - authorization)
|   |   |-- Admin.java        (extends Account)
|   |   +-- Staff.java        (extends Account)
|   |-- factory/              <-- Object Creation
|   |   +-- EmployeeFactory.java (role -> correct subclass)
|   |-- tools/                <-- Utility
|   |   |-- Validator.java     (regex + constraint rules)
|   |   +-- Inputter.java      (input validation helper)
|   |-- business/             <-- Business Layer
|   |   |-- EmployeeManager.java (9 features + file I/O)
|   |   +-- AuthService.java    (login + account list)
|   +-- main/                 <-- Presentation Layer
|       +-- Main.java          (login + menu loop + access check)
+-- employees.txt              (employee data file)
```

4. Class Descriptions

4.1. Model Layer

Payable.java (Interface)

Declares the calculateSalary() contract, demonstrating polymorphism: clients depend on Payable, not concrete classes.

Employee.java (Abstract Class)

Abstract base class holding the common state and defining the abstract methods getRole() and roleAllowance() for subclass-specific behaviour (Abstraction).

Member	Type	Description
-id	String (private)	Unique identifier (E\d{3})
-name / -status	String	Name; active/inactive
-baseSalary / -bonus	double	Base salary (> 0); bonus (>= 0)
-workingDays	int	Working days (0-26)
+getRole()	abstract String	Role name - Polymorphism
+roleAllowance()	abstract double	Role allowance - Polymorphism
+calculateSalary()	double	Template salary formula (uses

		roleAllowance()
+isActive()	boolean	True if status is active
+toDataLine() / toString()	String	File format / table display
+equals() / hashCode()	boolean / int	Compare by employee ID

Developer / Tester / Manager / HR (extends Employee)

Four subclasses that inherit Employee and override roleAllowance() and getRole() (Inheritance + Polymorphism).

Subclass	roleAllowance()
Developer	0.10 * baseSalary
Tester	0.05 * baseSalary
Manager	0.20 * baseSalary + 200
HR	0.08 * baseSalary

Account.java (Abstract Class) - Authorization

Abstract base class representing a user account. Holds username/password and defines the abstract methods getRole() and getAllowedFeatures(). The canAccess(feature) method is polymorphic, so the menu never needs an if-else on the role.

Member	Type	Description
-username / -password	String (private)	Login credentials (Encapsulation)
+getRole()	abstract String	Role name - Polymorphism
+getAllowedFeatures()	abstract Set<Integer>	Allowed feature numbers - Polymorphism
+canAccess(feature)	boolean	True if the feature is allowed for this role
+matches(user, pass)	boolean	Credential check for login

Admin / Staff (extends Account)

Subclass	getAllowedFeatures()
Admin	{1,2,3,4,5,6,7,8,9} - full access
Staff	{1,5,6,7,9} - read-only

4.2. Factory Layer

EmployeeFactory.java

Centralizes object creation: maps a role string to the correct Employee subclass. Adding a new role only requires one new subclass and one factory branch (Open/Closed Principle).

4.3. Tools Layer

Validator.java

Centralizes all validation rules in one place (Pattern Recognition): ID_REGEX (E\d{3}), ROLES, STATUSES, and range checks.

Inputter.java

Utility class for keyboard input with built-in validation loops:

Method	Description
inputInt(prompt, min, max)	Input integer within [min, max]
inputByRegex(prompt, regex, err)	Input string matching a regex
inputInSet(prompt, allowed)	Input a value within an allowed set
inputNonEmpty / inputPositiveDouble / inputNonNegativeDouble	Constrained inputs
inputOptional(prompt)	Optional input (Enter to keep current value)
confirmYesNo(prompt)	Y/N confirmation

4.4. Business Layer

EmployeeManager.java

Manages the List<Employee> and all 9 features. Uses a dirty flag to track unsaved changes.

Method	Function
load()	Read employees.txt; skip & count invalid lines safely
add() / update() / remove()	CRUD with validation; confirm before remove
searchByAttribute()	Search by ID / Name / Role / Status (substring)
monthlyPayroll()	Sum salary of active employees (polymorphic)
display() / save()	Table display / write back to file
quit()	Confirm saving before exit (dirty flag)

AuthService.java

Manages the List<Account> and the login process (max 3 attempts).

Method	Function
login()	Prompt credentials, authenticate, return the Account or null
authenticate(user, pass)	Find a matching account in the list

4.5. Presentation Layer

Main.java

Main entry point. Requires login first; then runs the 9-feature menu loop and checks currentUser.canAccess(choice) before executing each feature. Sets Locale.US for consistent number formatting.

5. How to Run the Program

5.1. System Requirements

- JDK 1.8 or higher (installed and JAVA_HOME configured)
- IDE: IntelliJ IDEA / Eclipse / NetBeans (IntelliJ recommended)
- OS: Windows / macOS / Linux

5.2. Running with IDE (IntelliJ IDEA)

1. Open Netbean -> File -> Open -> select the EmployeePayrollManagementSystem folder
2. Wait for the IDE to finish indexing all Java files
3. Open file src/main/Main.java
4. Right-click on Main.java -> select "Run 'Main.main()'"
5. The program will run in the IDE's Console window

[ảnh] Screenshot of IntelliJ IDEA right-clicking Main.java and selecting Run 'Main.main()'.

5.3. Running with Command Line

Step 1: Open Terminal and navigate to the project directory.

Step 2: Compile all Java files:

```
javac -encoding UTF-8 -d out -sourcepath src src/main/Main.java
```

Step 3: Run the program:

```
java -cp out main.Main
```

5.4. Default Accounts

Username	Password	Role
admin	admin123	Admin (full access)
staff	staff123	Staff (read-only)

5.5. Startup / Login Screen

On startup the system asks the user to log in, then displays the menu. Functions outside the current role are marked [locked].

```
===== LOGIN =====
Username: admin
Password: admin123
Login successful. Welcome admin (Admin).
===== EMPLOYEE PAYROLL MANAGEMENT SYSTEM =====
Logged in as: admin (Admin)
-----
```



```

=====
==== LOGIN ====
user&pass of admin: admin&admin123
user&pass of staff: staff&staff123
Username: admin
Password: admin1234
Invalid username or password. Attempts left: 2
Username: admin
Password: admin123
Login successful. Welcome admin (Admin).
==== EMPLOYEE PAYROLL MANAGEMENT SYSTEM ====
Logged in as: admin (Admin)
-----
1. Load employee data from file
2. Add a new employee
3. Update employee information
4. Remove an employee by ID

```

6. Feature Usage Guide

6.0. Login & Authorization

On startup, enter a username and password. After 3 failed attempts the program exits. Once logged in, the menu shows all 9 features but marks the ones outside your role as [locked]; selecting a locked feature shows "Access denied".

```

-----
1. Load employee data from file
2. Add a new employee
3. Update employee information
4. Remove an employee by ID
5. Search employees by attribute
6. Calculate monthly payroll
7. Display employee list
8. Save data to file
9. Quit program
=====
Choose 1-9: |

```

6.1. Feature 1: Load employee data from file

Enter 1 -> Reads employees.txt line by line. Invalid lines (wrong field count, bad number, failed validation, or duplicate ID) are skipped safely and counted. Example: "Loaded 11 record(s), skipped 3 invalid line(s)."

```
=====
Choose 1-9: 1

Loaded 11 record(s), skipped 0 invalid line(s).

===== EMPLOYEE PAYROLL MANAGEMENT SYSTEM =====
```

6.2. Feature 2: Add a new employee (Admin only)

Enter 2 -> Input the following fields (each validated; invalid input is re-prompted):

6. Employee ID (Exxx) - format validated, duplicates rejected
7. Name - non-empty
8. Role - Developer/Tester/Manager/HR
9. Base salary - must be positive
10. Working days - 0 to 26
11. Bonus - ≥ 0
12. Status - active/inactive

If all fields are valid -> "Employee added successfully."

```
=====
Choose 1-9: 2

ID (Exxx): E328
Name: Khuong
Role (Developer/Tester/Manager/HR): HR
Base salary (> 0): 10000
Working days (0-26): 22
Bonus (>= 0): 5
Status (active/inactive): active
Employee added successfully.
```

6.3. Feature 3: Update employee information (Admin only)

Enter 3 -> Input Employee ID -> Update role, salary, working days, bonus, status. Press Enter to keep the current value. Changing the role re-creates the object as the correct subclass via the factory.

```
Choose 1-9:
ID to update: E328
Press Enter to keep the current value.
Role [HR]:
Base salary [10000]:
Working days [22]:
Bonus [5]:
Status [active]:
Updated successfully.
```

6.4. Feature 4:

Remove an employee by ID (Admin only)

Enter 4 -> Input Employee ID -> Confirm (Y/N). Employee removed, or "This employee does not exist!" / "Cancelled."

```
ID to remove: E328
Are you sure you want to remove Khuong? (Y/N): Y
Removed successfully.
```

6.5. Feature 5:

Search employees by attribute

Enter 5 -> Choose attribute (1.ID 2.Name 3.Role 4.Status) -> Input keyword -> Displays all matching employees (case-insensitive substring match).

```
Choose 1-9: 5

Search by: 1. ID 2. Name 3. Role 4. Status
Choose 1-4: 2
Keyword: thanh

ID | Name | Role | Base | Day | Bonus | Status | Salary
-----
E293 | thanh | Developer | 100 | 11 | 1 | active | 53.3
-----
```

6.6. Feature 6: Calculate monthly payroll

Enter 6 -> Displays each active employee with their computed salary (polymorphic calculateSalary()), then the TOTAL payroll. Inactive employees are skipped.

```
=====
Choose 1-9: 6
```

ID	Name	Role	Salary
E004	David Miller	Developer	1,430.0
E005	Emma Wilson	Tester	878.3
E006	Frank Brown	HR	1,168.8
E007	Grace Taylor	Developer	1,578.8
E008	Henry Anderson	Manager	2,847.7
E010	Jack Moore	Developer	1,890.0
E011	Karen Lee	HR	971.7
E013	Nina Clark	Developer	1,609.2
E015	Brooke Guest	Manager	2,961.2
E293	thanh	Developer	53.3

6.7. Feature 7: Display employee list

Enter 7 -> Displays all employees in a formatted table (ID | Name | Role | Base | Day | Bonus | Status | Salary).

```
=====
Choose 1-9: 7
```

ID	Name	Role	Base	Day	Bonus	Status	Salary
E004	David Miller	Developer	1300	21	250	active	1,430.0
E005	Emma Wilson	Tester	950	20	100	active	878.3
E006	Frank Brown	HR	1100	22	150	active	1,168.8
E007	Grace Taylor	Developer	1250	24	300	active	1,578.8
E008	Henry Anderson	Manager	2200	19	600	active	2,847.7
E009	Ivy Thomas	Tester	980	18	120	inactive	847.5
E010	Jack Moore	Developer	1400	26	350	active	1,890.0
E011	Karen Lee	HR	1050	20	80	active	971.7
E012	Leo Martin	Manager	2100	17	500	inactive	2,493.1
E013	Nina Clark	Developer	1350	23	280	active	1,609.2
E015	Brooke Guest	Manager	2050	25	380	active	2,961.2
E293	thanh	Developer	100	11	1	active	53.3

6.8. Feature 8: Save data to file (Admin only)

Enter 8 -> Writes all employees back to employees.txt in the 7-field format. Example: "Data saved to 'employees.txt' (N record(s))."

```
Data saved to 'employees.txt' (12 record(s)).
```

6.9. Feature 9: Quit program

Enter 9 -> If unsaved changes are detected (dirty flag), prompts "Save before exit? (Y/N)". Then exits with "Goodbye!".

```
=====
Choose 1-9: 9

Goodbye!

Process finished with exit code 0
|
```

7. OOP Concepts Applied

OOP Concept	Application	Specific Example
Abstraction	Abstract base classes define common interfaces	Employee.roleAllowance(); Account.getAllowedFeatures(); interface Payable
Inheritance	Subclasses reuse base state and behaviour	Developer extends Employee; Admin extends Account
Polymorphism	Overridden methods drive behaviour without if-else	calculateSalary() uses roleAllowance(); menu uses canAccess()
Encapsulation	Private fields with controlled access	Employee.setWorkingDays() (0-26); Account hides password
Open/Closed	Extend without modifying existing logic	Add a new role/account = one new subclass only

8. Conclusion

8.1. Achievements

- Implemented all 9 features for employee management and payroll.
- Added a role-based authorization layer (login + Admin/Staff permissions).
- Applied OOP principles to both the employee model and the authorization model.
- Strict validation: regex (ID), value ranges, role/status sets, unique ID.
- Polymorphic salary calculation and polymorphic access control (no if-else on role).
- Monthly payroll counts active employees only.
- Safe file loading: malformed lines are skipped and reported, never crashing.

8.2. Challenges & Solutions

Challenge	Solution
Different allowance per role without if-else	Abstract Employee + roleAllowance() overridden in each subclass
Role-based access without if-else on role	Abstract Account + getAllowedFeatures() overridden; menu calls canAccess()
Salary formula not provided by the assignment	Self-defined a transparent formula and justified it
Reading a file that may contain erroneous data	try-catch + field-count + per-field validation; skip & count invalid lines
Inconsistent number formatting across locales	Locale.setDefault(Locale.US)